

Formation Engine Methodology

XYZ File Formats

VSEPR-SIM Atomistic Engine

Formats: `.xyz` `.xyza` `.xyzc` `.xyzf`

Edition 2 — Expanded Reference

VSEPR-SIM Project

Version 4.0.0 — April 13, 2026

Abstract

All VSEPR-SIM molecular I/O uses plain-text, line-oriented files. Four formats share a common structural grammar; each adds fields to the preceding one. This document is the expanded second-edition reference for the `.xyz`, `.xyza`, `.xyzc`, and `.xyzf` file formats, including grammar specifications, field semantics, coordinate conventions, checkpoint and trajectory usage, bounding-box encoding, implementation mapping, and typical observable ranges.

Contents

1	Overview	3
1.1	Unit Conventions	3
1.2	Coordinate System	3
2	Standard XYZ (<code>.xyz</code>)	3
2.1	Grammar	3
2.2	Comment Line Convention	4
2.3	Example: Water	4
2.4	Parsing Notes	4
3	Extended XYZ (<code>.xyza</code>)	4
3.1	Grammar	4
3.2	Property Columns	4
3.3	Properties Declaration Syntax	4
3.4	Example: H ₂ O with Charges and Velocities	5
3.5	Charge Semantics	5

4	Checkpoint XYZ (.xyzc)	5
4.1	Header Block	5
4.2	Header Field Semantics	6
4.3	Restart Protocol	6
4.4	Checkpoint vs. Trajectory	6
5	Formation XYZ (.xyzf)	6
5.1	Grammar	6
5.2	Comment Line Convention (Multi-Frame)	7
5.3	Frame Independence	7
5.4	Typical Uses	7
6	Bounding Box and Periodic Boundary Conditions	7
6.1	Comment-Line Encoding	7
6.2	Box Vector Interpretation	8
6.3	Minimum Image Convention	8
7	Bond Auto-Detection	8
7.1	Design Rationale	8
8	Implementation Mapping	8
8.1	Reader API Pattern	9
8.2	Writer API Pattern	9
9	Typical Energy and Observable Ranges	9
10	Integration with the Identity–State Framework	9
11	Format Comparison Matrix	10
12	Error Handling and Validation	10
13	Cross-References	10

§1 Overview

The VSEPR-SIM atomistic engine reads and writes molecular geometry, dynamics, and checkpoint data using four progressively richer plain-text formats. Each format extends the previous one by appending per-atom or per-frame metadata while preserving backward compatibility with simple line-oriented parsers.

Table 1: Format hierarchy.

Extension	Name	Use case
.xyz	Standard XYZ	Static geometry snapshot
.xyza	Extended XYZ	Charges, velocities, forces
.xyzc	Checkpoint XYZ	Restartable MD state
.xyzf	Formation XYZ	Multi-frame trajectory

1.1 Unit Conventions

All formats share a single, non-negotiable unit system:

Table 2: Unit system (all formats).

Quantity	Unit	SI equivalent
Coordinates	Å (angstrom)	10^{-10} m
Energy	kcal/mol	4.184×10^3 J/mol
Velocity	Å/fs	10^5 m/s
Force	kcal/(mol Å)	6.947×10^{13} N/mol
Charge	elementary e	1.602×10^{-19} C
Time	fs	10^{-15} s
Temperature	K	kelvin

1.2 Coordinate System

Right-handed Cartesian (x, y, z) in Å. Origin is arbitrary; the conventional choice is near the centre of mass. Coordinate precision: six decimal places ($\Rightarrow$ resolution ≈ 0.1 pm).

§2 Standard XYZ (.xyz)

2.1 Grammar

```
<N>
<comment>
<sym> <x> <y> <z>
... (N lines)
```

- **Line 1:** atom count N (integer ≥ 1).
- **Line 2:** free-text comment (may be empty).
- **Lines 3–($N+2$):** element symbol + three Cartesian coordinates, whitespace-separated.

2.2 Comment Line Convention

The comment line follows a structured-but-optional pattern:

```
<name> | E = <val> kcal/mol | <notes>
```

Fields are pipe-delimited. The energy key `E =` is recognised by the reader for energy-tracking contexts. Additional free-text notes are permitted after the second pipe.

2.3 Example: Water

```
3
H2O | E = -6.245 kcal/mol
O   0.000000  0.000000  0.119262
H   0.000000  0.763239 -0.477049
H   0.000000 -0.763239 -0.477049
```

2.4 Parsing Notes

- Element symbols are case-sensitive: `He` is helium, `HE` is rejected.
- Trailing whitespace on atom lines is ignored.
- Blank lines between the comment line and the first atom line are *not* permitted; the reader expects N consecutive atom lines starting at line 3.
- Coordinate fields must parse as IEEE 754 double-precision floating-point values.

§3 Extended XYZ (.xyza)

Adds per-atom scalar and vector properties after the (x, y, z) fields.

3.1 Grammar

```
<N>
<comment> properties="<list>","<sym> <x> <y> <z> [<q> <vx> <vy> <vz> <fx>
<fy> <fz> <e>]
```

The `properties` key declares which optional columns are present. Column order is fixed; omission is permitted only from the right.

3.2 Property Columns

3.3 Properties Declaration Syntax

The `properties` value is a colon-separated list of keys:

Table 3: Extended property columns (fixed order).

Key	Symbol	Unit	Columns
charge	q_i	e	1
velocity	$\mathbf{v}_i$	$\text{\AA}/\text{fs}$	3
force	$\mathbf{F}_i$	$\text{kcal}/(\text{mol } \text{\AA})$	3
energy	ε_i	kcal/mol	1

```
properties="charge:velocity"
properties="charge:velocity:force:energy"
```

Unrecognised keys trigger a parse warning but do not abort the read.

3.4 Example: H₂O with Charges and Velocities

```
3
Water MD frame 1 properties="charge:velocity"
O  0.00  0.00  0.12 -0.834  0.00  0.00  0.00
H  0.00  0.76 -0.48  0.417  0.12  0.08 -0.03
H  0.00 -0.76 -0.48  0.417 -0.12  0.08 -0.03
```

3.5 Charge Semantics

The charge field q_i represents the effective partial charge used in Coulomb interactions. It is *not* an oxidation state. For Drude-polarisable models, q_i is the core charge; the Drude particle charge is stored separately in the polarisation subsystem.

§4 Checkpoint XYZ (.xyzc)

Extends .xyza with a simulation-state header block required to restart an interrupted MD run without loss of trajectory continuity.

4.1 Header Block

The header block appears *before* the atom-count line, delimited by CHECKPOINT / END_CHECKPOINT markers:

```
CHECKPOINT
step      <integer>
time      <float> fs
dt        <float> fs
T_target  <float> K
seed      <integer>
box       <ax> <ay> <az>
END_CHECKPOINT
<N>
<comment>
```

```
<atom lines with charge + velocity + force>
```

4.2 Header Field Semantics

Table 4: Checkpoint header fields.

Field	Type	Meaning
<code>step</code>	integer	MD step number at checkpoint
<code>time</code>	float	Accumulated simulation time (fs)
<code>dt</code>	float	Integrator timestep (fs)
<code>T_target</code>	float	Thermostat set-point (K); 0 = NVE ensemble
<code>seed</code>	integer	RNG seed for reproducible continuation
<code>box</code>	3 floats	Simulation box side lengths (Å); omit line entirely for open boundary

4.3 Restart Protocol

When the engine loads a `.xyzc` file:

1. The header is parsed; integrator state restored (step, time, dt, thermostat target, RNG state).
2. The atom block is read as a standard `.xyza` frame.
3. Neighbour lists and bond tables are reconstructed from coordinates; they are *never* serialised into the checkpoint.
4. If a `box` line is present, PBC is activated.

4.4 Checkpoint vs. Trajectory

A `.xyzc` file contains exactly one frame. For multi-frame output, use `.xyzf` (§5).

§5 Formation XYZ (`.xyzf`)

A multi-frame format: one or more `.xyz` or `.xyza` blocks concatenated in a single file. Used for minimisation traces, NVT trajectories, and reaction paths.

5.1 Grammar

```
<N>          <- frame 1
<comment>
<atom lines>
<N>          <- frame 2
<comment>
<atom lines>
...
```

Each frame is self-contained and independently parseable.

5.2 Comment Line Convention (Multi-Frame)

The comment line of frame k typically records the step index and instantaneous energy:

```
Step 0 | E = -12.30 kcal/mol | FIRE iter
Step 1 | E = -13.11 kcal/mol | FIRE iter
```

5.3 Frame Independence

Each frame must be independently parseable:

- Atom count N may vary between frames.
- The `properties` declaration may differ between frames.
- Readers must re-parse the header of each frame; caching the first frame's column layout globally is a bug.

5.4 Typical Uses

Table 5: Common `.xyzf` applications.

Application	Comment line pattern
FIRE minimisation trace	Step <k> E = <val> kcal/mol FIRE iter
NVT trajectory dump	Step <k> E = <val> kcal/mol T = <T> K
Reaction path (NEB)	Image <k> E = <val> kcal/mol dE = <dE>
Conformer scan	Conf <k> E = <val> kcal/mol <dihedral>

§6 Bounding Box and Periodic Boundary Conditions

When PBC are active, box geometry is stored in the `.xyzc` header (§4) or as comment-line keys in any format.

6.1 Comment-Line Encoding

```
Lattice="ax 0 0 0 ay 0 0 0 az" pbc="T T T"
```

- Box vectors in Å, flattened 3×3 row-major matrix. Orthorhombic: diagonal entries only; triclinic: full matrix.
- `pbc` flags: T = periodic, F = open.
- Compatible with ASE Extended XYZ convention.

6.2 Box Vector Interpretation

For an orthorhombic cell with side lengths a_x, a_y, a_z :

$$\mathbf{L} = \begin{pmatrix} a_x & 0 & 0 \\ 0 & a_y & 0 \\ 0 & 0 & a_z \end{pmatrix} \quad (1)$$

The cell volume is $V = \det(\mathbf{L}) = a_x \cdot a_y \cdot a_z$.

6.3 Minimum Image Convention

Under PBC, the engine applies the minimum-image convention:

$$\Delta r_\alpha = r_\alpha^{(j)} - r_\alpha^{(i)} - L_\alpha \cdot \text{round}\left(\frac{r_\alpha^{(j)} - r_\alpha^{(i)}}{L_\alpha}\right), \quad \alpha \in \{x, y, z\} \quad (2)$$

where L_α is the box length along axis α .

§7 Bond Auto-Detection

Bond connectivity is inferred from geometry, never stored in the file. The `detect_bonds` function uses per-element covalent radii:

$$d_{ij} < s \cdot (r_{\text{cov}}(Z_i) + r_{\text{cov}}(Z_j)) \implies \text{bond}(i, j) \quad (3)$$

where s is the scale factor (default $s = 1.2$).

7.1 Design Rationale

Bonds are derived data, not ground truth. Stale bond tables are a silent-corruption vector. The §0 container philosophy formalises this: caches never become truth.

§8 Implementation Mapping

Table 6: Source files implementing the XYZ format family.

File	Role
<code>include/io/xyz_format.hpp</code>	Public API: <code>XYZReader</code> , <code>XYZWriter</code> , <code>XYZAReader</code> , <code>XYZAWriter</code>
<code>src/io/xyz_format.cpp</code>	Reader / writer implementation
<code>include/thermal/xyzc_format.hpp</code>	Checkpoint reader/writer declarations
<code>src/thermal/xyzc_format.cpp</code>	Checkpoint reader/writer implementation

All reader and writer types reside in namespace `vsepr::io`.

8.1 Reader API Pattern

```
// Standard XYZ
auto frame = vsepr::io::XYZReader::read("molecule.xyz");
// Extended XYZ
auto frame = vsepr::io::XYZAReader::read("dynamics.xyza");
// Checkpoint
auto state = vsepr::io::XYZCReader::read("restart.xyzc");
// Formation (multi-frame)
auto traj = vsepr::io::XYZFReader::read_all("trajectory.xyzf");
// or frame-by-frame:
vsepr::io::XYZFReader reader("trajectory.xyzf");
while (auto frame = reader.next()) { /* ... */ }
```

8.2 Writer API Pattern

```
vsepr::io::XYZWriter::write("output.xyz", atoms, comment);
vsepr::io::XYZAWriter::write("output.xyza", atoms, props, comment);
vsepr::io::XYZCWriter::write("checkpoint.xyzc", state);
vsepr::io::XYZFWriter writer("trajectory.xyzf", /*append=*/true);
writer.write_frame(atoms, comment);
```

§9 Typical Energy and Observable Ranges

Table 7: Typical observable magnitudes for sanity checking.

Observable	Typical range	Unit
Total bonded energy	-10^2 to -10^5	kcal/mol
Relative conformer energy	0.1 – 10	kcal/mol
Activation barrier	5 – 50	kcal/mol
Force (unrelaxed atom)	1 – 100	kcal/(mol Å)
Force convergence criterion	< 0.01	kcal/(mol Å)
Velocity (300 K, C atom)	~ 0.02	Å/fs
Partial charge (O in water)	-0.82 to -0.84	e
Partial charge (H in water)	+0.41 to +0.42	e

These ranges serve as first-pass sanity checks. Values outside these bounds in production output should trigger audit warnings.

§10 Integration with the Identity–State Framework

The XYZ formats serve as the serialisation layer for the particle identity vector $\mathbf{I}_i$ defined in §0 of the Formation Engine methodology.

$\mathcal{K}_{\text{geom}}$ maps to the lattice matrix from the `Lattice` key or `box` header. $\mathcal{K}_{\text{comp}}$ is reconstructed at load from element composition and geometry. The source-layer flag $F_K \in \{Z, A, C\}$ maps $Z \rightarrow .xyz$, $A \rightarrow .xyza$, $C \rightarrow .xyzc$.

Table 8: Identity vector components and their file-format encoding.

Component	Symbol	File encoding
Nuclear identity	Z_i	Element symbol (looked up to Z)
Mass identity	A_i	Implicit via Z_i
Charge participation	Q_i	charge column in .xyza
Structural role	Σ_i	Not serialised; inferred at load
Stability class	Λ_i	Not serialised; computed post-load
Provenance memory	Θ_i	Comment-line metadata

§11 Format Comparison Matrix

Table 9: Feature comparison across formats.

Feature	.xyz	.xyza	.xyzc	.xyzf
Atom count + coordinates	✓	✓	✓	✓
Partial charges		✓	✓	opt.
Velocities		✓	✓	opt.
Forces		✓	✓	opt.
Per-atom energy		✓	✓	opt.
Integrator state			✓	
Thermostat / RNG			✓	
Box geometry (header)			✓	
Box geometry (comment key)	✓	✓	✓	✓
Multi-frame				✓

§12 Error Handling and Validation

Readers enforce these validation rules at parse time:

1. **Atom count mismatch.** Fewer than N atom lines $\Rightarrow$ `XYZParseError::INCOMPLETE_FRAME`.
2. **Unknown element.** Unrecognised symbol $\Rightarrow$ `XYZParseError::UNKNOWN_ELEMENT`.
Two-letter symbols must match the periodic table (case-sensitive).
3. **Malformed coordinates.** Non-numeric field $\Rightarrow$ `XYZParseError::BAD_COORDINATE`.
4. **Column count mismatch.** Declared `properties` implies k columns but line has fewer: warn and zero-fill (fail-soft for velocities/forces; fail-hard for coordinates).
5. **Checkpoint integrity.** Missing `CHECKPOINT/END_CHECKPOINT` block $\Rightarrow$ `XYZCParseError::NO_HE`

§13 Cross-References

Table 10: Related documentation.

Document	Relevance
<code>docs/FILE_FORMATS.md</code>	Prose description of all I/O formats
<code>section0_identity_state.pdf</code>	Formal particle/cell/world definitions
<code>section5_integration.tex</code>	MD timestep and integrator details
<code>ALPHA_MODEL_BOOKLET.tex</code>	Alpha model (Drude charge coupling)
<code>section_fuzzy_ball_model.tex</code>	Coarse-grained bead visualisation